Operating Systems - Lecture Notes

1. Introduction to Operating Systems

An operating system (OS) is system software that manages computer hardware and provides a stable environment in which application programs can run. It has two broad jobs. First, it acts as a resource manager, sharing the CPU, main memory, storage devices and input/output devices fairly and efficiently among competing programs. Second, it acts as an abstraction layer, hiding hardware details behind simpler concepts such as files, processes and sockets.

Modern processors support at least two privilege levels. In user mode, ordinary application code runs with restricted rights and cannot execute privileged instructions or touch hardware directly. In kernel mode, the operating system kernel runs with full access to all hardware and memory. This split protects the system: a buggy application can crash itself but cannot overwrite the kernel or another program's memory.

When an application needs an OS service, such as reading a file, it makes a system call. The system call triggers a controlled switch from user mode to kernel mode, the kernel performs the work on behalf of the program, and control returns to user mode. Common kernel designs include monolithic kernels, where most services run inside the kernel, and microkernels, where only the bare minimum runs in kernel mode and other services run as user-level servers.

2. Processes and Threads

A process is a program in execution. Besides the program code, a process has its own address space, open files, and a program counter. The operating system tracks every process using a data structure called the process control block (PCB). A PCB typically stores the process ID, the current process state, the saved program counter and CPU registers, memory-management information such as page tables, scheduling information like priority, and a list of open files and I/O devices.

During its life a process moves between states: new, ready, running, waiting (blocked), and terminated. A ready process is waiting for the CPU, a running process is currently executing, and a waiting process is blocked until an event such as I/O completion occurs. Moving the CPU from one process to another is called a context switch. The kernel saves the state of the old process into its PCB and loads the saved state of the next one. Context switching is pure overhead because no useful user work is done while it happens.

A thread is a lightweight unit of execution inside a process. Threads of the same process share code, heap and open files, but each thread has its own stack, program counter and registers. Creating a thread and switching between threads is cheaper than doing the same for processes, but a bug in one thread can corrupt data shared with the others, so synchronisation is needed.

3. CPU Scheduling

The CPU scheduler picks which process in the ready queue gets the CPU next. Common algorithms are described below.

First-Come First-Served (FCFS) runs processes in arrival order. It is simple but suffers from the convoy effect: many short processes wait behind one long CPU-bound process, which increases average waiting time.

Shortest Job First (SJF) selects the process with the smallest next CPU burst. It gives the minimum average waiting time for a given set of processes, but the burst length must be predicted and long jobs may starve.

Round Robin (RR) is designed for time-sharing systems. Each process receives a small slice of CPU time called the time quantum, typically 10 to 100 milliseconds. When the quantum expires, the process is preempted and placed at the back of the ready queue. If the quantum is very large, RR behaves like FCFS; if it is very small, too much time is wasted on context switches.

Priority scheduling gives the CPU to the process with the highest priority. Its main problem is starvation, where low-priority processes may wait indefinitely. The standard remedy is aging, which gradually raises the priority of a process the longer it waits in the ready queue, so every process eventually runs.

Multilevel feedback queues use several ready queues with different priorities and let processes move between queues based on their observed behaviour.

4. Deadlocks

A deadlock is a situation in which a set of processes are all blocked, each holding a resource and waiting for a resource held by another process in the set. A deadlock can occur only if four conditions hold at the same time, known as the Coffman conditions: mutual exclusion (a resource can be used by only one process at a time), hold and wait (a process holds at least one resource while waiting for others), no preemption (resources cannot be forcibly taken away), and circular wait (a closed chain of processes exists in which each waits for a resource held by the next).

Deadlock prevention works by making sure at least one of the four conditions can never hold, for example by requiring processes to request all resources at once or by imposing a global ordering on resource types to break circular wait. Deadlock avoidance is less restrictive: the system examines each request and grants it only if the resulting state is safe. The Banker's algorithm is the classic avoidance method. It keeps track of the maximum demand, current allocation and available resources, and grants a request only if there is still some order in which all processes can finish.

Deadlock detection allows deadlocks to happen, then looks for them using a resource-allocation graph or similar matrices. Recovery can be done by terminating one or more processes or by preempting resources. Many general-purpose operating systems simply ignore the problem, an approach called the ostrich algorithm, because deadlocks are rare.

5. Memory Management

Paging divides physical memory into fixed-size blocks called frames and logical memory into blocks of the same size called pages. A page table maps each page number to a frame number. Paging removes external fragmentation, although the last page of a process may waste some space, which is called internal fragmentation.

Because the page table lives in main memory, each memory access would otherwise need two memory reads. The translation lookaside buffer (TLB) is a small, fast hardware cache that stores recent page-to-frame translations, so most address translations avoid the extra access to the page table. The fraction of lookups found in the TLB is the hit ratio.

In demand paging, a page is loaded only when it is first used. If a program references a page that is not in memory, a page fault occurs and the OS fetches the page from disk. When no free frame is available, a page replacement algorithm chooses a victim. FIFO replaces the oldest page, Optimal replaces the page that will not be used for the longest time (it cannot be implemented in practice and is used as a benchmark), and LRU replaces the page that has not been used for the longest time.

FIFO can show Belady's anomaly: for some reference strings, giving the process more frames actually increases the number of page faults. LRU and Optimal do not suffer from this anomaly. If a process does not have enough frames, it spends more time swapping pages than executing, a state called thrashing.